FINANCE WITH PHIL · INTERNAL GUIDE

# Social Media Analytics Setup Guide

How to set up the FWP analytics dashboard from a blank Mac to a live, PIN-gated site that tracks Instagram, TikTok, YouTube, Threads and LinkedIn, with model-labeled comment sentiment. Every command here is the one actually used on Phil's machine.

**Live site**
phillipkaraya.github.io/fwp-analytics-dashboard

**Repository**
github.com/phillipkaraya/fwp-analytics-dashboard

**Written**
September 5, 2026 · matches commit e8d7cfe

CONTENTS



## 1. What you end up with

A static Next.js site, exported to plain HTML and JSON and hosted free on GitHub Pages behind a PIN. It has five tabs: **Overview** (followers, a 30/90/365-day window of views, posts, likes and comments, platform strip, reach and engagement by platform, follower history), **Post Analysis** (every post, filterable and sortable), **Comments** (7,999 comments with sentiment and question flags), **Insights** (viral posts, hooks, hashtags, superfans, questions worth answering, cross-posting) and the **Content Vault** (every post filed by topic).

The data behind it comes from a Python scraper that drives the Chrome you are already signed in to, so there are no API keys for the social platforms and nothing to renew. The only external service is Groq's free tier, used once to label comment sentiment and then only for new comments.

**Platforms**

Instagram, TikTok, YouTube, Threads, LinkedIn. Threads and LinkedIn publish no view count, so their reach is measured in likes and their engagement against followers.

**Refresh cadence**

Run the scraper by hand when new posts go out. One run per day is plenty and adds one point to the follower history.

**Cost**

Zero. GitHub Pages, GitHub Actions, and Groq's free tier. No paid APIs, no server.

**Time to set up**

About an hour, plus a 15 to 30 minute first scrape and a 25 minute first sentiment pass that run on their own.

## 2. Before you start: the checklist

| YOU NEED | WHY | CHECK |
|---|---|---|
| macOS with Google Chrome | The scraper reads the platforms through your signed-in Chrome. | Chrome opens and you can sign in to all five platforms. |
| Node 20 and pnpm 10 | Builds the site. The deploy workflow uses the same versions. | `node -v` and `pnpm -v` |
| Python 3.9 or newer | Runs the scraper and analyzer. Standard library plus one package. | `python3 --version` |
| git and the GitHub CLI | Push to the repo and set the one repository secret. | `gh auth status` shows phillipkaraya |
| Groq account (free) | Optional. Labels comment sentiment with a model; without it a lexicon does the job. | A key from console.groq.com stored in Keychain (Step 6). |
| `secret-sync` | Phil's Keychain resolver. Keys reach a process as environment variables and never touch a file. | `secret-sync doctor` |

RULE

The dashboard PIN and every API key stay out of the repository. The repo is public. The PIN is stored as a SHA-256 hash in a GitHub secret, the Groq key lives in Keychain, and the scraper never copies cookies out of Chrome.

## 01 Get the code and install

```
cd ~/projects
git clone https://github.com/phillipkaraya/fwp-analytics-dashboard.git
cd fwp-analytics-dashboard
pnpm install
pip3 install websocket-client
```

`websocket-client` is the only Python dependency. It carries the Chrome DevTools Protocol connection. Everything else in `scrape/` is standard library, which is why it runs on the Python that ships with macOS.

## 02 Run it locally and set the PIN

```
pnpm dev
```

Open `http://localhost:3000`. You will meet the PIN wall. In development the gate accepts a built-in fallback PIN; for anything you deploy, set your own. The site never stores the PIN itself, only its SHA-256 hash, which is compared in the browser with the Web Crypto API and remembered for the tab session.

Make a hash without the PIN ever appearing in your shell history:

```
read -s PIN; printf '%s' "$PIN" | shasum -a 256 | cut -d' ' -f1; unset PIN
```

For local development put that hash in `.env.local` (gitignored):

```
NEXT_PUBLIC_DASHBOARD_PIN_HASH=paste-the-64-character-hash-here
```

Keep the hash handy. Step 8 puts the same value into a GitHub secret so the live site uses it too.

> GOOD TO KNOW
> The dev server reads `public/data/*.json` on every request, so after any scrape you just reload. If styles ever look stale after edits, stop the server and remove `.next/dev` and `.next/cache`; Next 16's persistent dev cache occasionally serves an old stylesheet.

## 03 Chrome with remote debugging

The scraper does not log in anywhere. It connects to a Chrome that is already running with the DevTools Protocol enabled on port 9222, opens its own tabs, reads the pages as you would see them, and closes only the tabs it opened. That Chrome must be signed in to Instagram, TikTok, YouTube, Threads and LinkedIn.

On Phil's Mac the `chrome-cdp` shell function does this. It is safe to call when Chrome is already listening; it just prints the version. If nothing is listening yet, it quits Chrome and relaunches it with the

debugging port, so run it before you start working in Chrome, not in the middle of something.

```
chrome-cdp
curl -s http://127.0.0.1:9222/json/version
```

On another Mac without that helper, quit Chrome and launch it once with the flag. The profile is your normal one, so your logins come along:

```
open -a "Google Chrome" --args --remote-debugging-port=9222
```

Then sign in to each of the five platforms in that Chrome window. Sessions on Instagram, TikTok, YouTube and Threads last a long time. LinkedIn's expires by design every few weeks; when it does, the LinkedIn scrape stops with a message telling you to sign in again, and the other four platforms carry on.

> NEVER
>
> Do not relaunch Chrome on a different port to scrape, and do not close tabs you did not open. The scraper coordinates with other automation through `ccbrowser` when it is present, claims a tab lease for the run, and refuses to start when the Mac is critically low on memory.

## 04 Point the scraper at the right accounts

All handles live in one file, `scrape/handles.py`. Change them there and nowhere else.

```
HANDLES = {
    "instagram": "phillip.karaya",
    "tiktok": "phillip.karaya",
    "youtube": "@phillip.karaya",
    "threads": "phillip.karaya",
    "linkedin": "phillip-karaya",
}
YOUTUBE_CHANNEL_ID = "UCHTb2yDMGoptvmgxh49UniA"
INSTAGRAM_USER_ID = "5251656103"
```

Two of these are stable identifiers rather than handles. YouTube is scraped by channel ID so a handle rename cannot break it; the handle is only used for display and links. The Instagram user ID is the numeric suffix of every Instagram post ID. When a handle changes (TikTok and YouTube both moved from financewithphil to phillip.karaya in 2026), update the handle here and the next run picks it up.

## 05 The first data pull

With Chrome listening and signed in, run a master sweep once. It pages every profile to the end and writes one JSON file per platform.

```
cd ~/projects/fwp-analytics-dashboard
python3 scrape/run.py --full
```

Budget 15 to 30 minutes. TikTok is the slow one because it scrolls the page. You will see one line per platform as each finishes, like `linkedin: +16 new, 16 total, followers 1714 (67s)`, then the analyzer runs and prints its summary. After that, the everyday command is the incremental run, which stops each platform after it has seen eight posts in a row that it already knows:

```
python3 scrape/run.py                      # incremental, all five platforms
python3 scrape/run.py --platforms linkedin  # one platform only
python3 scrape/run.py --analyze-only        # rebuild derived JSON without scraping
```

| FILE WRITTEN | BY | WHAT IT HOLDS |
|---|---|---|
| `public/data/<platform>_posts.json` | the platform scrapers | every post with views, likes, comments, shares, date, caption, type |
| `public/data/scrape_state.json` | `run.py` | per-platform status, last run time, follower counts |
| `public/data/analytics.json` | `analyze.py` | every Insights section and the comment sentiment counts |
| `public/data/content_vault.json` | `analyze.py` | posts grouped by topic for the Content Vault |
| `public/data/follower_history.json` | `analyze.py` | one follower snapshot per day a scrape ran, appended, never rewritten |

HOW EACH PLATFORM IS READ

**Instagram** hooks the page's own GraphQL timeline responses (direct API calls get throttled). **TikTok** reads the video tiles while scrolling; dates are decoded from the video ID. **YouTube** uses the channel page data plus innertube continuations for shorts and videos. **Threads** hooks the profile GraphQL query and keeps only posts by the handle (a reload can land on the For You feed). **LinkedIn** reads the rendered activity-feed cards from your own profile; the feed is server-rendered so no network hook can see it, and the scraper never reads messaging.

## 06 Comment sentiment with Groq

Comments are labeled by `scrape/sentiment.py` whenever the analyzer runs. Each comment gets a `sentiment` of positive, neutral or negative and a separate `isQuestion` flag, so a comment can be both positive and a question. The labels are written back into `comments.json`, which makes that file the cache: a rerun only labels comments that have no current label.

There are two labelers. The **model** path uses Groq's free tier and is the accurate one. The **lexicon** path is an emoji-aware, word-boundary matcher tuned to this audience (congrats, fire, goat, salute, facts) that runs without any key in under a second and fills in whenever the model is unavailable.

### Store the key once

Create a key at console.groq.com, then store it in Keychain. This is the only command that ever sees the key, it prompts with hidden input, and you type it once:

```
secret-sync set GROQ_API_KEY
```

### Run the labeling pass

```
secret-sync run GROQ_API_KEY -- python3 scrape/run.py --analyze-only
```

The resolver hands the key to the process as an environment variable and nothing else. The first full pass over 8,000 comments takes about 320 model calls. Each run is capped at 400 calls, and Groq's free models allow roughly 8,000 tokens a minute each, so expect the log to show 429 rate-limit counts per model. That is normal: a rate-limited model sits out for the seconds Groq asks for while the others keep working. If the first pass ends with some rows still on the lexicon, run the same command again; the second pass only sends those rows and takes about 50 calls.

The progress line looks like this:

```
sentiment: 725 labeled by model so far, 30 calls, 1.7s per call, errors
{'http429:gpt-oss-120b': 9, ...}
sentiment: 1249 by model, 0 by lexicon, 6750 already current, 51 calls
```

> RESULT ON THIS CORPUS
>
> Positive 5,300 (66%), neutral 2,187 (27%), negative 512 (6%), 559 questions, all 7,999 labeled by the model. The old labels had 81% neutral because they came from a substring matcher with no emoji handling.

If you change the prompt or the lexicon, bump `SENTIMENT_VERSION` in `scrape/sentiment.py` and every comment is relabeled on the next run. A Groq 403 with error code 1010 is a Cloudflare user-agent block, not a dead key; the scraper already sends a browser user agent.

## 07 Topics for the Content Vault

`scrape/categories.json` maps a topic slug to a label and a keyword list. Matching is whole-word and case-insensitive against title, caption and hashtags, a post can land in several topics, and anything that matches nothing goes to *Other*. Add or move keywords, then rebuild without scraping:

```
python3 scrape/run.py --analyze-only
```

No code change is needed to add a topic. The Vault's hero shows how many posts are still unfiled so you can see whether a new keyword helped.

## 08 Deploy to GitHub Pages

The repository ships with a GitHub Actions workflow at `.github/workflows/deploy.yml`. On every push to `main` it installs with pnpm, builds the static export with the base path set to the repository name, adds a `.nojekyll` marker and publishes to Pages. You do three things once:

1. In the repository on GitHub, open **Settings** → **Pages** and set **Source** to **GitHub Actions**.
2. Add the PIN hash from Step 2 as a repository secret. The workflow reads it as `DASHBOARD_PIN_HASH`:

   ```
   read -s HASH; gh secret set DASHBOARD_PIN_HASH --body "$HASH"; unset HASH
   ```

3. Push. The first push to `main` runs the workflow; the site is live about a minute later at `https://<user>.github.io/<repo>/`.

Every later deploy is just a push. To confirm a deploy landed, watch the run and then read a data file straight from the live site with a cache-busting query:

```
gh run list --limit 1
curl -s "https://phillipkaraya.github.io/fwp-analytics-
dashboard/data/scrape_state.json?cb=$(date +%s)" | head -c 400
```

> WHY STATIC
>
> There is no server and no database. The browser fetches JSON files that were committed to the repo, so the site is as fresh as the last push and costs nothing to host. That is also why the scraper runs on your Mac rather than in the cloud: it needs your logged-in Chrome.

---

## The day-to-day routine

When new posts have gone out, or once a day when you want a follower point:

```
cd ~/projects/fwp-analytics-dashboard
python3 scrape/run.py
git add public/data && git commit -m "Refresh data" && git push
```

That is the whole loop. The incremental run takes a couple of minutes, the analyzer rebuilds every derived file, the push redeploys. A few habits keep it healthy:

- **LinkedIn once a day at most.** It drives your real session with human-like pauses. If it reports a login wall, sign in to linkedin.com in the shared Chrome and rerun with `--platforms linkedin`.

- **Master sweep occasionally, not on a schedule.** `--full` refreshes views and likes on every old post and takes up to half an hour.
- **Sentiment refresh only when comments change.** The comment corpus is still the March 2026 snapshot; when a comment scraper exists, the same labeling command will label only the new rows.
- **Follower history needs a scrape on a new day** to add a point. Re-running the analyzer on old data never invents one.

## How the numbers are computed

| NUMBER | RULE |
|---|---|
| Overview window | The shortest of 30, 90 or 365 days that contains at least one post; deltas compare against the window before it. The hero says which window it picked and why. |
| Reach | Views where the platform publishes them. Threads and LinkedIn publish none, so their reach number is likes and the charts label the bar "(likes)". Likes are never summed into a views total. |
| Engagement rate | Likes plus comments plus shares, divided by views. For Threads and LinkedIn, divided by followers instead, computed at scrape time and stored on the post. |
| Averages in Insights | Only posts with a view count above zero are averaged, so viewless platforms never drag them down. |
| Follower deltas | From the last two points in `follower_history.json`. A platform missing from an older snapshot (LinkedIn before September 2026) starts its line at its first point. |
| Comment sentiment | Labels from the model, lexicon as fallback, stored per comment. The UI never re-classifies; a missing label counts as neutral. |
| Questions | The `isQuestion` flag, independent of sentiment. Ranked by likes; the reply state in the snapshot is unreliable and is never used. |
| Dates | Date-only strings are treated as local dates. Parsing them as UTC shows every date a day early in Eastern time, which is the bug this rule prevents. |

## Troubleshooting

| SYMPTOM | CAUSE | FIX |
|---|---|---|
| LinkedIn run stops with "login or checkpoint wall" | LinkedIn's session cookie expired, which it does by design. | Sign in to linkedin.com in the shared Chrome, then `python3 scrape/run.py --platforms linkedin`. |

| | | |
|---|---|---|
| Instagram returns 429 or an HTML page | Instagram throttles scripted API calls even from a logged-in tab. | Already handled: the scraper reads the page's own GraphQL responses. If it recurs, wait ten minutes and rerun. |
| Threads suddenly has hundreds of posts by other people | A reload landed on the For You feed. | Already handled: posts are filtered by handle. Revert the file with git if an old version ever merged them. |
| TikTok "account not found" or YouTube 404 | The handle was renamed. | Update `scrape/handles.py`. YouTube is keyed by channel ID and keeps working. |
| Groq 403 with error code 1010 | Cloudflare blocks non-browser user agents. | Not a dead key. The scraper sends a browser UA; if you call the API elsewhere, do the same. |
| Groq log full of 429 counts | Free models allow about 8,000 tokens a minute; a 25-comment batch nearly fills it. | Normal. The run cools that model and continues. Run the command a second time if rows remain on the lexicon. |
| Scraper refuses to start, "memory critical" | `ccbrowser mem` found the Mac short on memory. | Close heavy apps or tabs, then rerun. The check exists so a scrape can never freeze the machine. |
| Dev server shows old styles after an edit | Next 16's persistent dev cache. | Stop the server, `rm -rf .next/dev .next/cache`, start again. |
| Live site shows old data after a push | The Pages run has not finished, or the browser cached the JSON. | `gh run list --limit 1`, then hard-refresh. The data status bar in the header shows the last scrape time. |
| Charts look empty in an automated browser | A hidden tab pauses animation frames, so Recharts never draws. | Not a bug. Check the DOM or make the tab visible. |

## File map

```
app/                       one route per tab (static export)
components/
  charts/                  BarChart, LineChart, DoughnutChart, Section, Numbered, Badge
  layout/                  AppShell, Nav, AuthGate, PinWall, DataStatusBar, PageHero
  overview/ posts/ comments/ insights/ vault/
lib/
  data.ts                  typed loaders for public/data/*.json
  derive.ts                totals, byPlatform, hasViews, reach, followerDeltas, sentiment
  format.ts                fmt, fmtDate, numeralShift, platformLabel, platformShort
  auth.ts                  SHA-256 PIN check
  types.ts                 the JSON shapes
scrape/
  run.py                   CLI: scrape, then analyze
  analyze.py               analytics.json, content_vault.json, follower_history.json
  sentiment.py             comment labels: Groq round-robin with cooldown, lexicon fallback
  categories.json          topic keywords (editable)
  handles.py               handles and stable IDs
  cdp.py                   Chrome DevTools client, own-tab discipline
  platforms/               instagram.py tiktok.py youtube.py threads.py linkedin.py
public/data/               the committed JSON the site reads
.github/workflows/deploy.yml   build and publish on push to main
```

## Rules that keep this safe

- **No secrets in the repo, ever.** The PIN is a hash in a GitHub secret. The Groq key is in Keychain and reaches the process only through `secret-sync run`. Before every commit: `git status --short | grep -iE '\.env|secret|key|token'` should print nothing.
- **The scraper uses your Chrome, it does not copy it.** Cookies and login data are never copied out of the profile. The scraper opens its own tabs and closes only those.
- **LinkedIn messaging is off limits.** The scraper reads only the public activity feed and the profile follower count. Nothing from messaging is read, stored or logged.
- **Measure what a platform publishes.** Threads and LinkedIn have no view counts; the dashboard shows likes as reach instead of showing zeros or blanks.
- **Phone width is part of done.** Every card must fit a 375px screen, and every chart must label every category. The project note records the exact checks.